

lab5_report

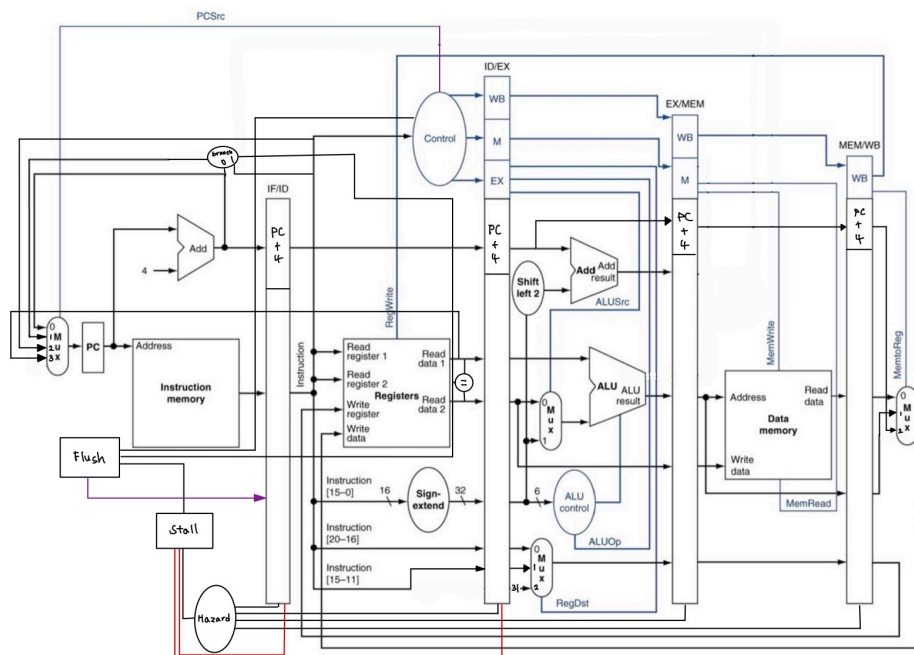
: 2022056562 서민용, 2023003227 이한주

1. Overall structure

1-1. Pipelined CPU Structure

- 이번 lab5에서는 기존 Multi Cycle CPU의 구조를 확장하여 5-stage Pipelined CPU를 구현하였습니다.
- 기존에는 하나의 instruction이 2~5단계의 cycle에 걸쳐 수행되었으나, 이번에는 각각의 instruction들이 서로 다른 stage에서 동시에 진행할 수 있도록 pipeline register를 추가하였습니다.
- Data Hazard와 Control Hazard를 피하기 위한 stall, bubble 및 flush logic도 구현하였습니다.

1-2. CPU Microarchitecture



- 기존 Multi Cycle CPU와 달리 Pipelined CPU는 추가적인 Microarchitectural Register들이 많이 필요합니다.
- 각 Stage의 사이에 이를 배치하여 각 단계의 명령어, 필요한 Control Signal 및 데이터 등을 전달합니다.
- 가장 핵심적인 부분은 각각의 명령어에 해당하는 Control Signal을 함께 전달하는 것으로, 이후 Stage에서도 올바른 동작을 수행하기 위해 꼭 필요합니다.

- 서로 다른 Stage의 정보들을 구분하기 위하여 Pipeline Register들은 IFID, IDEX, EXMEM, MEMWB 접두사를 사용합니다.

2. Implementation

2-1. CTRL.v

- opcode와 funct를 입력받아 총 8개의 control signal을 생성하는 combinational logic unit으로 구현했습니다.

```
RegWrite
MemtoReg
MemWrite
PCSource
SignExtend
ALUSrc
ALUOp
RegDst
```

- 기존 Multi Cycle CPU와는 달리 FSM의 state로 signal을 구분하지 않고 Decode stage에서 해석된 명령어에 해당하는 signal을 한번에 생성한 후, pipeline register를 통해 이후 stage로 전달됩니다.
- 즉, lab3 Single Cycle CPU의 CTRL.v와 유사한 구조를 이어받았으며, 앞서 설계한 CPU diagram을 기반으로 하여 적절한 Control Signal들을 생성하도록 변형했습니다.
- Signal 값은 opcode에 따라서 상위 모듈인 CPU.v 에 위치한 MUX의 올바른 입력값이 되도록 설정해 줍니다.

2-2. CPU.v

- 상단에는 PC 및 Pipeline register가 선언되어있습니다.

```
reg [31:0] PC;

reg        IFID_Valid;
reg [31:0] IFID_PC;
reg [31:0] IFID_Inst;

reg        IDEX_Valid;
reg        IDEX_RegWrite;
...
```

- IF Stage에서는 현재 PC를 instruction memory의 address로 사용합니다.

```
assign inst_addr = PC;
```

- 다음 PC 값은 네 가지 경우에 따라 나뉩니다.

- 일반적인 경우 - PC + 4
- branch - id_branch_target (2-3에서 후술)
- j, jal - id_jump_target (2-3에서 후술)
- jr - rd_data1

```
case (id_PCSrc)
    2'd0: PC_next = PC + 4;
    2'd1: PC_next = branch_taken ? id_branch_target : (PC + 4);
    2'd2: PC_next = id_jump_target;
    2'd3: PC_next = rd_data1;
endcase
```

- 특히, branch와 jump instruction에 대해서 early resolution을 적용하여 Decode stage에서 모두 처리하도록 구현했습니다.

2-3. Early Branch / Jump Resolution

- BEQ와 BNE는 RF에서 읽은 두 값을 직접 비교하여 taken 여부를 판단합니다.

```
assign branch_taken =
    id_valid_inst && !id_halt &&&
    (((opcode == `OP_BEQ) && (rd_data1 == rd_data2)) ||
    ((opcode == `OP_BNE) && (rd_data1 != rd_data2)));
```

- 기존처럼 EX Stage에서 ALU 연산 결과를 이용해 branch를 판단할 경우 더 많은 cycle에 필요하고, Control hazard의 패널티가 커지기도 하고, Jump instruction과 함께 ID Stage에서 다음 PC를 판단할 수 있게 구현하는 것이 더 간단하므로 Early resolution을 적용하였습니다.
- 따라서, PC_next 주소 계산 시 IDEX_* Signal 대신 현재 Decode stage의 rd_data1, rd_data2, IFID_PC, immi, immj를 사용했습니다.
- Branch target address는 sign extension 및 shift left 2를 수행하여 다음과 같이 계산합니다.

```
assign id_branch_target = IFID_PC + {{14{immi[15]}}, immi, 2'b00};
```

- Jump target은 shift left 2를 적용하고, 현재 PC의 상위 4비트를 가져옵니다.

```
assign id_jump_target = {IFID_PC[31:28], immj, 2'b00};
```

- J, JAL은 rd_data1을 다음 PC로 사용합니다.

diagram

2-4. Control Hazard, Flush

- 이 CPU의 Branch Prediction 정책은 always not-taken입니다.

- ID Stage에서 branch나 jump 명령어가 확인되기 이전에는 IF Stage에서 PC + 4를 fetch하게 됩니다.
- 따라서 ID Stage에 위치한 명령어가 branch이면서 taken이거나, jump 명령어인 경우 IF Stage에서 fetch한 명령어는 잘못된 명령어이므로 flush 하도록 구현했습니다.

```
assign control_taken =
    id_valid_inst && !id_halt &&
    ((id_PCSrc == 2'd2) ||
    (id_PCSrc == 2'd3) ||
    ((id_PCSrc == 2'd1) && branch_taken));

assign control_flush = control_taken && !stall_active;
```

- control_taken은 ID Stage에서 분기가 확정되었음을 알리는 값입니다.
- control_flush 값을 별도로 두어 stall이 존재할 경우 이를 우선적으로 수행하도록 구현했습니다.

```
if (control_flush) begin
    IFID_Valid <= 1'b0;
    IFID_PC <= 32'd0;
    IFID_Inst <= 32'd0;
end
```

- flush인 경우, Valid bit를 0으로, ID Stage의 PC 값과 Inst 값을 0으로 설정합니다.

2-5. Data Hazard, Stall

- Data hazard 발생 시 올바른 횟수만큼 stall을 삽입하도록 구현했습니다.

Stall when

- (rs(IR_{ID}) == dest_{EX}) && use_rs(IR_{ID}) && RegWrite_{EX} // 3 Cycles
- (rs(IR_{ID}) == dest_{MEM}) && use_rs(IR_{ID}) && RegWrite_{MEM} // 2 Cycles
- (rs(IR_{ID}) == dest_{WB}) && use_rs(IR_{ID}) && RegWrite_{WB} // 1 Cycle
- (rt(IR_{ID}) == dest_{EX}) && use_rt(IR_{ID}) && RegWrite_{EX} // 3 Cycles
- (rt(IR_{ID}) == dest_{MEM}) && use_rt(IR_{ID}) && RegWrite_{MEM} // 2 Cycles
- (rt(IR_{ID}) == dest_{WB}) && use_rt(IR_{ID}) && RegWrite_{WB} // 1 Cycle

- 위 사진에 나타나있는 Hazard들은 별도의 모듈 HAZARD.v에서 처리합니다.

```
always @(*) begin
    stall = 2'b00;

    if (((rs == dest_EX) && use_reg[0] && RegWrite_EX) ||
```

```

        ((rt == dest_EX) && use_reg[1] && RegWrite_EX))) begin
            stall = `STALL3;
        end
    else if (((rs == dest_MEM) && use_reg[0] && RegWrite_MEM) ||
              ((rt == dest_MEM) && use_reg[1] && RegWrite_MEM))) begin
            stall = `STALL2;
        end
    else if (((rs == dest_WB) && use_reg[0] && RegWrite_WB) ||
              ((rt == dest_WB) && use_reg[1] && RegWrite_WB))) begin
            stall = `STALL1;
        end
    end
end

```

- instruction 종류에 따라서 실제로 사용하는 register를 구분함으로써 불필요한 stall을 줄일 수 있었습니다.

```

`OP_RTYPE: begin
    case (funct)
        `FUNCT_JR: use_reg = 2'b01;
        `FUNCT_SLL, `FUNCT_SRA, `FUNCT_SRL: use_reg = 2'b10;
        default: use_reg = 2'b11;
    endcase
end

```

2-6. Stall, Pipeline Registers

- Stall이 발생하면 IFID pipeline register는 유지되어야 합니다.
- IDEX pipeline register에는 bubble을 삽입했습니다.

```

if (stall_active) begin
    PC <= PC;
    IFID_Valid <= IFID_Valid;
    IFID_PC <= IFID_PC;
    IFID_Inst <= IFID_Inst;

    IDEX_Valid <= 1'b0;
    IDEX_Halt <= 1'b0;
    IDEX_RegWrite <= 1'b0;
    IDEX_MemWrite <= 1'b0;
    IDEX_PCSource <= 2'b00;
end

```

- 이 구조에서는 EXMEM, MEMWB pipeline register는 계속 진행하도록 하여 IF와 ID Stage만을 정지시키고, EX, MEM, WB Stage의 instruction들은 모두 실행되어 hazard가 해소되게 됩니다.

2-7. Valid Bit, Halt

- 본 구조에서는 bubble 삽입을 위해 instruction address에 0을 삽입했습니다.
- 하지만 이는 halt instruction과 동일하기 때문에 bubble과 halt를 구분하는 과정이 필요합니다.

```
IFID_Valid
IDEX_Valid
EXMEM_Valid
MEMWB_Valid

IDEX_Halt
EXMEM_Halt
MEMWB_Halt
```

- 이를 해결하기 위해 각 pipeline register에 valid bit 및 halt 여부를 추가하였고, halt signal의 결정 방법을 MEM/WB stage에서 valid bit와 함께 결정하도록 변경하였습니다.

```
assign halt = MEMWB_Valid && MEMWB_Halt;
```

- 이로써 유효한 halt instruction만이 CPU를 종료시킬 수 있습니다.

2-8. Write-back

- WB Stage에서는 MEMWB pipeline register에 저장된 control signal을 기준으로 RF에 값을 기록합니다.
- 기록할 data는 MemtoReg에 따라 선택합니다.

```
case (MEMWB_MemtoReg)
    2'd0: RegData = MEMWB_Data;
    2'd1: RegData = MEMWB_ALUResult;
    2'd2: RegData = MEMWB_PC;
endcase
```

- RegData의 값은 다음 세 가지 경우로 구분됩니다.
 - lw - MEMWB_Data
 - ALU operations - MEMWB_ALUResult
 - jal - MEMWB_PC

3. Result

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 11185000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 4125000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 2535000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 17775000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5545000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 4995000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

```
minyongseo@MinYong-MacBook-Air HW05_HanJoo % ./a.out
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 12785000 (1ps)
minyongseo@MinYong-MacBook-Air HW05_HanJoo %
```

- 기존의 CPU 들과 달리 첫 pipelining structure을 도입시킨 결과라 그런지 테스트케이스마다 차이는 존재하지만 대략 1.8배에서 2배정도 wall clock time이 줄어든 모습입니다.
- 지금은 단순 always Not Taken 예측이라서 비교하여 inspection 할게 많진 않지만,
- 다음 lab6 때 Branch Predictor나 BTB가 도입된다면 Hit Ratio에 따른 wall clock time 추이를 보면 재미있을 듯 합니다.

4. Troubleshooting

- 기존 lab2에서, RF.v 의 combinational logic 관련하여 @(*)와 @(rd_addr1, rd_addr2) 중 후자를 선택했었습니다.
 - 그 이유는 Read timing 이 아니라 Write timing임에도 불구하고 reading이 되는 것을 GTKWave 로 파악했었기 때문입니다.
 - 물론 reading된 값을 사용하여 arch' state를 수정하지 않기 때문에, 문제는 없었으나 후자가 더 robust하다고 판단했었습니다.
 - lab2~4까지는 모두 한 싸이클 이내에 RF read만 일어나거나, RF write만 일어났기에, 문제가 전혀 없었습니다.
- 허나 지금은 pipelined multicycle CPU 이기 때문에, 한 싸이클 이내에 RF read와 write가 동시에 일어날 수 있습니다.
- 하여 수정된 arch' state의 값을 읽어야 할 수 있기 때문에 전자로 수정해주었더니 모든 테스트 케이스가 잘 돌아갔습니다.

```
// * <-> rd_addr1, rd_addr2 ...
// 기존 single/multi cycle 에서는 한 싸이클 안에 RF read(async) 와 RF
write(sync)가 같이
// 일어날 일이 없었어서 rd_addr1, rd_addr2로 해주어도 큰 문제가 없었음.
// But 지금은 한 싸이클 이내에 일어날 수 있어서 *로 해주어야함.
// 즉 예를 들어 WB 이후 ID 가 필요한 상황; forwarding 이 있었다면 internal forwarding인 상황 같은 경우 문제가 생긴다.
always @(*) begin
```

```
        rd_data1 = register_file[rd_addr1];  
        rd_data2 = register_file[rd_addr2];  
    end
```